1. When conducting the work of Lab 11, we conducted the test that uses the Central Limit Theorem even though the sample size was "small" (i.e., $n < 30$). It turns out, that how "far off" the t-test is can be computed using a first-order Edgeworth approximation for the error. Below, we will do this for the the further observations.

   (a) Boos and Hughes-Oliver (2000) note that

$$P(T \leq t) \approx F_Z(t) + \underbrace{\frac{\text{skew}}{\sqrt{n}} \frac{(2t^2+1)}{6} f_Z(t)}_{\text{error}},$$

   where $f_Z(\cdot)$ and $F_Z(\cdot)$ are the Gaussian PDF and CDF and skew is the skewness of the data. What is the potential error in the computation of the p-value when testing $H_0 : \mu_X = 0; H_a : \mu_X < 0$ using the zebra finch further data?

```
finch.dat <- read_csv("zebrafinches.csv")

## Rows:  25 Columns:  3
## -- Column specification --------------------------------------------------------
## Delimiter:  ","
## dbl (3):  closer, further, diff
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

n <- nrow(finch.dat)
#summarize the data
sum.farther.vals <- finch.dat |>
summarise(
  mean = mean(further),
  sd = sd(further),
  skew = skewness(further)
)
#compute t statistic
t.farther <- t.test(finch.dat$further, mu = 0, alternative = "less")

t.stat <- t.farther$statistic

fz <- dnorm(t.stat)

error <- ((sum.farther.vals$skew/sqrt(n))*((2*(t.stat)^2+1)/6)*fz)
error

##             t
## -1.226006e-13
```

   (b) Compute the error for t statistics from -10 to 10 and plot a line that shows the error across t. Continue to use the skewness and the sample size for the zebra finch further data.

```
  t.vals <- seq(-10, 10, length.out = 1000 )

FZ.vals <- dnorm(t.vals)

error.vals <- (sum.farther.vals$skew / sqrt(n)) * ((2 * t.vals^2 + 1) / 6) * FZ.vals

error.df <- tibble(
    t     = t.vals,
    error = error.vals
  )
```

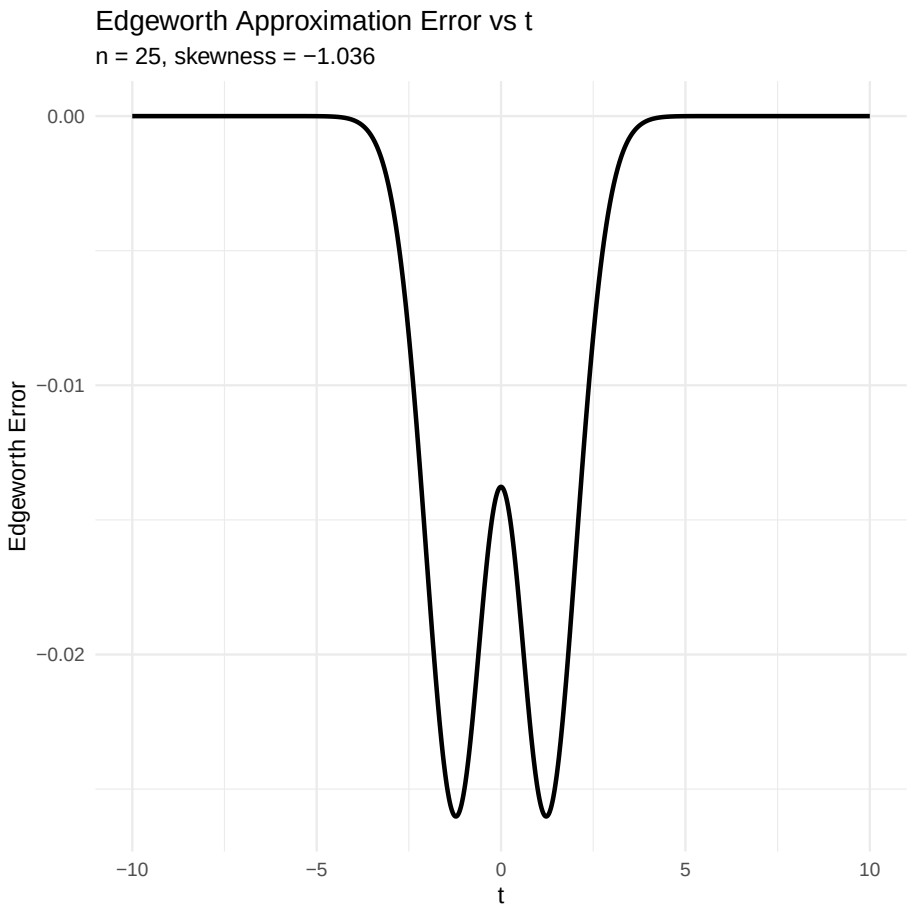

Figure 1: Edgeworth Appromixmation Error across t

(c) Suppose we wanted to have a tail probability within 10% of the desired $\alpha = 0.05$. Recall we did a left-tailed test using the further data. How large of a sample size would we need? That is, we need to solve the error formula equal to 10% of the desired left-tail probability:

$$0.10\alpha \overset{set}{=} \underbrace{\frac{\text{skew}}{\sqrt{n}} \frac{(2t^2+1)}{6} f_Z(t)}_{\text{error}},$$

which yields

$$n = \left(\frac{\text{skew}}{6(0.10\alpha)} (2t^2+1) f_Z(t) \right)^2.$$

```
alpha <- .05

theoretical.t.value  <- abs(qnorm(alpha))
fz                   <- dnorm(theoretical.t.value)


solved.sample.size <- (
abs(sum.farther.vals$skew)
/ (6 * (0.10 * alpha) * (2 * theoretical.t.value^2 + 1) * fz)
)^2
print(solved.sample.size)

## [1] 2725.099
```

2. Complete the following steps to revisit the analyses from lab 11 using the bootstrap procedure.

   (a) Now, consider the zebra finch data. We do not know the generating distributions for the closer, further, and difference data, so perform resampling to approximate the sampling distribution of the T statistic:

$$T = \frac{\bar{x}_r - 0}{s/\sqrt{n}},$$

   where $\bar{x}_r$ is the mean computed on the rth resample and s is the sample standard deviation from the original samples. At the end, create an object called `resamples.null.closer`, for example, and store the resamples shifted to ensure they are consistent with the null hypotheses at the average (i.e., here ensure the shifted resamples are 0 on average, corresponding to $t = 0$, for each case).

```
R <- 5000
n <- nrow(finch.dat)
s <- sd(finch.dat$closer)
resamples <- tibble(tstats = rep(NA, R))

for(i in 1:R){
curr.resample <- sample(finch.dat$closer,
                        size = n,
                        replace = TRUE)

resamples$tstats[i] <- (mean(curr.resample) - 0) / (s / sqrt(n))
}

# No longer a confidence interval for  this is for the T-statistic distribution
# Shift so H0 is true (centered at 0)
(delta <- mean(resamples$tstats))

## [1] 8.292631

resamples <- resamples |>
mutate(tstats.shifted = tstats - delta)

resamples.null.closer <- resamples$tstats.shifted

mean(resamples.null.closer) #this is close to zero

## [1] -2.543233e-16

# Use observed t-stat from original data
obs_t <- (mean(finch.dat$closer) - 0) / (s / sqrt(n))
low <- -abs(obs_t)
high <- abs(obs_t)

resamples |>
summarize(mean = mean(tstats.shifted),
          p.low = mean(tstats.shifted <= low),
          p.high = mean(tstats.shifted >= high))|>
mutate(p = p.low + p.high)

## # A tibble: 1 x 4
##        mean p.low p.high     p
##       <dbl> <dbl>  <dbl> <dbl>
## 1 -2.54e-16     0      0     0
```

```
R <- 5000
n <- nrow(finch.dat)
s <- sd(finch.dat$further)
resamples <- tibble(tstats = rep(NA, R))

for(i in 1:R){
curr.resample <- sample(finch.dat$further,
                        size = n,
                        replace = TRUE)

resamples$tstats[i] <- (mean(curr.resample) - 0) / (s / sqrt(n))
}

# Shift so H0 is true (centered at 0)
(delta <- mean(resamples$tstats))

## [1] -7.781087
```

```r
resamples <- resamples |>
mutate(tstats.shifted = tstats - delta)

resamples.null.further <- resamples$tstats.shifted

mean(resamples.null.further) #this is close to zero
```

```
## [1] 2.012931e-16
```

```r
# Use observed t-stat from original data
obs_t <- (mean(finch.dat$further) - 0) / (s / sqrt(n))
low <- -abs(obs_t)
high <- abs(obs_t)

resamples |>
summarize(mean = mean(tstats.shifted),
          p.low = mean(tstats.shifted <= low),
          p.high = mean(tstats.shifted >= high))|>
mutate(p = p.low + p.high)
```

```
## # A tibble: 1 x 4
##       mean p.low p.high     p
##      <dbl> <dbl>  <dbl> <dbl>
## 1 2.01e-16     0      0     0
```

```r
n <- nrow(finch.dat)
s <- sd(finch.dat$diff)
resamples <- tibble(tstats = rep(NA, R))

for(i in 1:R){
  curr.resample <- sample(finch.dat$diff,
                          size = n,
                          replace = TRUE)

  resamples$tstats[i] <- (mean(curr.resample) - 0) / (s / sqrt(n))
}

# Shift so H0 is true (centered at 0)
(delta <- mean(resamples$tstats))
```

```
## [1] 8.513471
```

```r
resamples <- resamples |>
  mutate(tstats.shifted = tstats - delta)

resamples.null.diff <- resamples$tstats.shifted

mean(resamples.null.diff) #this is close to zero
```

```
## [1] 6.811669e-16
```

```r
# Use observed t-stat from original data
obs_t <- (mean(finch.dat$diff) - 0) / (s / sqrt(n))
low <- -abs(obs_t)
high <- abs(obs_t)

resamples |>
  summarize(mean = mean(tstats.shifted),
            p.low = mean(tstats.shifted <= low),
            p.high = mean(tstats.shifted >= high))|>
  mutate(p = p.low + p.high)
```

```
## # A tibble: 1 x 4
##       mean p.low p.high     p
##      <dbl> <dbl>  <dbl> <dbl>
## 1 6.81e-16     0      0     0
```

(b) Compute the bootstrap p-value for each test using the shifted resamples. How do these compare to the t-test p-values?

Variable	Bootstrap p-value	t-test p-value
Closer	0.00	$\leq$0.0001
Further	0.00	$\leq$0.0001
Diff	0.00	$\leq$0.0001

Although the bootstrap p-value and the classical t-test p-value are not numerically identical, they are extremely close and lead to the same conclusion.

(c) What is the 5^{th} percentile of the shifted resamples under the null hypothesis? Note this value approximates $t_{0.05,n-1}$. Compare these values in each case.

```
df        <- nrow(finch.dat) - 1
alpha     <- 0.05

# Helper: obs t and null-shifted bootstrap t's
boot_tstats <- function(x, R = 5000, mu0 = 0) {
n  <- length(x)
s  <- sd(x)
t_stats <- replicate(R, {
  xs <- sample(x, n, TRUE)
  (mean(xs) - mu0)/(s/sqrt(n))
})
t_shift <- t_stats - mean(t_stats)
t_obs   <- (mean(x) - mu0)/(s/sqrt(n))
list(
  t_obs = t_obs,
  qs    = quantile(t_shift, c(.025, .05, .95, .975), names = FALSE)
)
}

# Compute for each variable
closer  <- boot_tstats(finch.dat$closer)
further <- boot_tstats(finch.dat$further)
diff    <- boot_tstats(finch.dat$diff)

# Theoretical cut-offs
t_left05    <- qt(alpha,    df)    # 5% left-tail
t_right95   <- qt(1-alpha,  df)    # 95% right-tail
t_two_low   <- qt(alpha/2,  df)    # 2.5% two-tail
t_two_high  <- qt(1-alpha/2,df)    # 97.5% two-tail
```

Variable	Observed t	Boot 5th percentile	t-test 5th percentile
Closer	8.30	-1.62	-1.71
Further	-7.78	-1.65	-1.71
Diff	8.51	-1.60	-1.71

Variable	Observed t	Boot 95th percentile	t-test 95th percentile
Closer	8.30	1.63	1.71
Further	-7.78	1.55	1.71
Diff	8.51	1.66	1.71

Variable	Boot 2.5th percentile	Boot 97.5th percentile	t-test 2.5th percentile	t-test 97.5th percentile
Closer	-1.93	1.97	-2.06	2.06
Further	-1.97	1.82	-2.06	2.06
Diff	-1.85	1.93	-2.06	2.06

Note that in all three cases the observed t statistic is the sam , but the choice of the percentile changes the decision boundary. Reporting only the 5^{th} percentile corresponds to a left-tailed test. For the closer data a right tailed test using the 95^{th} percentile would be more approporaite. For

the further data the 5^{th} percentile works. If there was no idea of a know direction before testing a two tailed cutoff with a 2.5^{th} percentile and a 97.5^{th} cutoff are most appropriate.

Then when looking at the differences between the bootstrap approach and the t-test we see what the percentiles are very similar.

(d) Compute the bootstrap confidence intervals using the resamples. How do these compare to the t-test confidence intervals?

Variable	Bootstrap 95% CI (BCa)	t-test 95% CI
Closer	[0.12, 0.19]	[0.12, 0.20]
Further	[-0.26, -0.16]	[-0.26, -0.15]
Diff	[0.29, 0.45]	[0.27, 0.45]

When comparing the bootstrap and the t-test confidence intervals we note that they are extremely close and some of the difference seen in the table above is due to rounding.

3. Complete the following steps to revisit the analyses from lab 11 using the randomization procedure.

(a) Now, consider the zebra finch data. We do not know the generating distributions for the closer, further, and difference data, so perform the randomization procedure.

```
#Randomization test for Closer
R   <- 10000
mu0 <- 0

#Build null distribution for closer
rand <- tibble(xbars = rep(NA, R))

# PREPROCESSING: shift the data to be mean 0 under H0
x.shift <- finch.dat$closer - mu0

# RANDOMIZE / SHUFFLE
for(i in 1:R){
curr.rand <- x.shift *
  sample(x      = c(-1, 1),
         size   = length(x.shift),
         replace = TRUE)

rand$xbars[i] <- mean(curr.rand)
}

# shift back
rand <- rand |> mutate(xbars = xbars + mu0)

#Compute the p value
(delta <- abs(mean(finch.dat$closer) - mu0))

## [1] 0.1562231

(low   <- mu0 - delta)  # mirror

## [1] -0.1562231

(high  <- mu0 + delta)  # observed

## [1] 0.1562231

p_value_closer <-
mean(rand$xbars <= low) +
mean(rand$xbars >= high)

p_value_closer  # randomization p value for closer

## [1] 0
```

```
#Randomization test for Further
R   <- 10000
mu0 <- 0

#Build null distribution for further
rand <- tibble(xbars = rep(NA, R))
```

```r
# PREPROCESSING: shift the data to be mean 0 under H0
x.shift <- finch.dat$further - mu0

# RANDOMIZE / SHUFFLE
for(i in 1:R){
  curr.rand <- x.shift *
    sample(x      = c(-1, 1),
     size   = length(x.shift),
     replace = TRUE)

  rand$xbars[i] <- mean(curr.rand)
}

# shift back
rand <- rand |> mutate(xbars = xbars + mu0)

# Compute two sided p value
(delta <- abs(mean(finch.dat$further) - mu0))

## [1] 0.2027244

(low   <- mu0 - delta)  # mirror

## [1] -0.2027244

(high  <- mu0 + delta)  # observed

## [1] 0.2027244

p_value_further <-
  mean(rand$xbars <= low) +
  mean(rand$xbars >= high)
```

```r
R   <- 10000
mu0 <- 0

#Build null distribution for "diff"
rand <- tibble(xbars = rep(NA, R))

# PREPROCESSING: shift the data to be mean 0 under H0
x.shift <- finch.dat$diff - mu0

# RANDOMIZE / SHUFFLE
for(i in 1:R){
curr.rand <- x.shift *
  sample(x      = c(-1, 1),
         size   = length(x.shift),
         replace = TRUE)

rand$xbars[i] <- mean(curr.rand)
}

# shift back
rand <- rand |> mutate(xbars = xbars + mu0)

#Compute two sided p value
(delta <- abs(mean(finch.dat$diff) - mu0))

## [1] 0.3589475

(low   <- mu0 - delta)  # mirror

## [1] -0.3589475

(high  <- mu0 + delta)  # observed

## [1] 0.3589475

p_value_diff <-
mean(rand$xbars <= low) +
mean(rand$xbars >= high)
```

(b) Compute the randomization test p-value for each test.

```r
p_value_closer

## [1] 0

p_value_further
```

```
## [1] 0

p_value_diff

## [1] 0
```

(c) Compute the randomization confidence interval by iterating over values of μ_0.
Hint: You can "search" for the lower bound from Q_1 and subtracting by 0.0001, and the upper bound using Q_3 and increasing by 0.0001. You will continue until you find the first value for which the two-sided p-value is greater than or equal to 0.05.

```
R             <- 5000
mu0.iterate   <- 0.0001
starting.point <- mean(finch.dat$closer)

#Lower bound search
mu.lower <- starting.point
repeat {
  rand <- tibble(xbars = rep(NA, R))
  x.shift <- finch.dat$closer - mu.lower

  for(i in 1:R){
    curr.rand     <- x.shift *
sample(c(-1, 1), length(x.shift), replace = TRUE)
    rand$xbars[i] <- mean(curr.rand)
  }
  rand <- rand |> mutate(xbars = xbars + mu.lower)

  delta <- abs(mean(finch.dat$closer) - mu.lower)
  low   <- mu.lower - delta
  high  <- mu.lower + delta
  p.val <- mean(rand$xbars <= low) +
    mean(rand$xbars >= high)

  if (p.val < 0.05) break
  mu.lower <- mu.lower - mu0.iterate
}

#Upper bound search
mu.upper <- starting.point
repeat {
  rand <- tibble(xbars = rep(NA, R))
  x.shift <- finch.dat$closer - mu.upper

  for(i in 1:R){
    curr.rand     <- x.shift *
sample(c(-1, 1), length(x.shift), replace = TRUE)
    rand$xbars[i] <- mean(curr.rand)
  }
  rand <- rand |> mutate(xbars = xbars + mu.upper)

  delta <- abs(mean(finch.dat$closer) - mu.upper)
  low   <- mu.upper - delta
  high  <- mu.upper + delta
  p.val <- mean(rand$xbars <= low) +
    mean(rand$xbars >= high)

  if (p.val < 0.05) break
  mu.upper <- mu.upper + mu0.iterate
}

ciR_closer <- c(mu.lower, mu.upper)
ciR_closer  # randomization 95 CI for closer

## [1] 0.1180231 0.1946231
```

```
R             <- 5000
mu0.iterate   <- 0.0001
starting.point <- mean(finch.dat$further)

#Lower bound search
mu.lower <- starting.point
repeat {
  rand <- tibble(xbars = rep(NA, R))
  x.shift <- finch.dat$further - mu.lower

  for(i in 1:R){
curr.rand     <- x.shift *
  sample(c(-1, 1), length(x.shift), replace = TRUE)
```

```
rand$xbars[i] <- mean(curr.rand)
  }
  rand <- rand |> mutate(xbars = xbars + mu.lower)

  delta <- abs(mean(finch.dat$further) - mu.lower)
  low   <- mu.lower - delta
  high  <- mu.lower + delta
  p.val <- mean(rand$xbars <= low) +
mean(rand$xbars >= high)

  if (p.val < 0.05) break
  mu.lower <- mu.lower - mu0.iterate
}

#Upper bound search
mu.upper <- starting.point
repeat {
  rand <- tibble(xbars = rep(NA, R))
  x.shift <- finch.dat$further - mu.upper

  for(i in 1:R){
curr.rand     <- x.shift *
  sample(c(-1, 1), length(x.shift), replace = TRUE)
rand$xbars[i] <- mean(curr.rand)
  }
  rand <- rand |> mutate(xbars = xbars + mu.upper)

  delta <- abs(mean(finch.dat$further) - mu.upper)
  low   <- mu.upper - delta
  high  <- mu.upper + delta
  p.val <- mean(rand$xbars <= low) +
mean(rand$xbars >= high)

  if (p.val < 0.05) break
  mu.upper <- mu.upper + mu0.iterate
}

ciR_further <- c(mu.lower, mu.upper)
ciR_further  # randomization 95 CI for further

## [1] -0.2560244 -0.1506244
```

```
R             <- 5000
mu0.iterate   <- 0.0001
starting.point <- mean(finch.dat$diff)

#Lower bound search
mu.lower <- starting.point
repeat {
  rand <- tibble(xbars = rep(NA, R))
  x.shift <- finch.dat$diff - mu.lower

  for(i in 1:R){
    curr.rand     <- x.shift *
sample(c(-1, 1), length(x.shift), replace = TRUE)
    rand$xbars[i] <- mean(curr.rand)
  }
  rand <- rand |> mutate(xbars = xbars + mu.lower)

  delta <- abs(mean(finch.dat$diff) - mu.lower)
  low   <- mu.lower - delta
  high  <- mu.lower + delta
  p.val <- mean(rand$xbars <= low) +
    mean(rand$xbars >= high)

  if (p.val < 0.05) break
  mu.lower <- mu.lower - mu0.iterate
}

#Upper bound search
mu.upper <- starting.point
repeat {
  rand <- tibble(xbars = rep(NA, R))
  x.shift <- finch.dat$diff - mu.upper

  for(i in 1:R){
    curr.rand     <- x.shift *
sample(c(-1, 1), length(x.shift), replace = TRUE)
    rand$xbars[i] <- mean(curr.rand)
  }
```

```
  rand <- rand |> mutate(xbars = xbars + mu.upper)

  delta <- abs(mean(finch.dat$diff) - mu.upper)
  low   <- mu.upper - delta
  high  <- mu.upper + delta
  p.val <- mean(rand$xbars <= low) +
    mean(rand$xbars >= high)

  if (p.val < 0.05) break
  mu.upper <- mu.upper + mu0.iterate
}

ciR_diff <- c(mu.lower, mu.upper)
ciR_diff  # randomization 95 CI for diff

## [1] 0.2752475 0.4446475
```

4. **Optional Challenge:** In this lab, you performed resampling to approximate the sampling distribution of the T statistic using

$$T = \frac{\bar{x}_r - 0}{s/\sqrt{n}}.$$

I'm curious whether it is better/worse/similar if we computed the statistics using the sample standard deviation of the resamples (s_r), instead of the original sample (s)

$$T = \frac{\bar{x}_r - 0}{s_r/\sqrt{n}}.$$

   (a) Perform a simulation study to evaluate the Type I error for conducting this hypothesis test both ways.

   (b) Using the same test case(s) as part (a), compute bootstrap confidence intervals and assess their coverage – how often do we 'capture' the parameter of interest?

References

Boos, D. D. and Hughes-Oliver, J. M. (2000). How large does n have to be for z and t intervals? *The American Statistician*, 54(2):121–128.